

Social Robot Behavior API

This document outlines the behavior interface for all social robots in the CARES lab. The behavior interface specifies how to command the robot to perform behaviors. Each behavior consists of one or more actions that the robot must execute. The actions are composed together in an Action Script.

The API is called via a webservice using POST for the body of the message. The URL for the webservice may be specific to the type of robot.

Each robot may also need a separate interface to start the necessary software components. This currently is not being standardized across the various robots.

Action Script

An Action Script describes a sequence of actions to be performed towards a particular intent. It has the following structure:

```
{
  "intent": <intent>,
  "description": <any text>,
  "actionList": [
    {
      "name": <action name>,
      "args": < list, of, argument, values >
      "metadata": {
        "trace": < list, of, argument, values >
      }
    },
    {
      "name": <action name>,
      "args": < list, of, argument, values >
      "metadata": {
        "trace": < list, of, argument, values >
      }
    },
    ...
  ]
}
```

Explanation of fields

intent: The value for the intent should specify the communicative intent of the robot's behavior. This value is currently not used by any of the robots, but it still needs to be present.

description: Any text may be entered in the description. The purpose of this field is to provide human-readable information about the action script.

actionList: The value is a list of actions. The actions will be executed in the order that they are listed. Each action has the following fields:

name: The name of the action. See below for the list of supported actions.

args: A list of values to be used as parameters to the action. Each action specifies the expected list of arguments.

metadata: A container for data that does not affect how the action is executed. The robot may use this data for logging purposes.

trace: A trace of the reasoning steps used to infer a given action.

An example is the following:

```
{
  "intent": "Example",
  "description": "This is an example of looking and speaking",
  "actionList": [
    {
      "name": "LookInDirection",
      "args": ["center"]
    },
    {
      "name": "SetEyes",
      "args": ["happy"]
    },
    {
      "name": "SayText",
      "args": ["Hello, my name is Isabelle."]
    },
    {
      "name": "Pause",
      "args": ["1500"]
    },
    {
      "name": "SetEyes",
      "args": ["default"]
    }
  ]
}
```

```
    },  
    {  
      "name": "LookInDirection",  
      "args": ["upperRight"]  
    }  
  ]  
}
```

Supported Actions

The following is a specification of each of the actions that may appear in an Action Script. Due to the capabilities and modalities available in each robot, the execution of each of these actions may vary. However, the intent is for the execution of the actions to be as similar as possible from robot to robot.

SayText

The SayText action instructs the robot to speak the given text. It is assumed that each robot has text-to-speech (TTS) capabilities.

List of arguments:

1. A string representing what the robot will say.

Example:

```
{
    "name": "SayText",
    "args": ["Hello World"]
}
```

Note: Future developments will allow the SayText action to take an unspecified amount of arguments. All arguments after the first are assumed to be values that will be inserted into the string at the appropriate location. The result should be something similar to string interpolation or printf.

LookInDirection

The LookInDirection action instructs the robot to look in a particular direction. The directions are described qualitatively and not with specific coordinates. The specified direction is relative to the robot's perspective (e.g., looking left means looking to the robot's left).

List of arguments:

1. A direction, which must be one of the following:
 - upperLeft
 - up
 - upperRight
 - left
 - center
 - right
 - lowerLeft

- down
- lowerRight

Example:

```
{
    "name": "LookInDirection",
    "args": ["upperRight"]
}
```

PointAt

The PointAt action instructs the robot to point in a general direction. The directions are described qualitatively and not with specific coordinates. The specified direction is relative to the robot's perspective (e.g., pointing left means pointing to the robot's left).

List of arguments:

1. A direction, which must be one of the following:
 - upperLeft
 - upwards
 - upperRight
 - left
 - straightOut
 - right
 - lowerLeft
 - downwards
 - lowerRight
 - default (each robot should have its own default position)
2. Which arm is to do the pointing, which must be one of the following:
 - left
 - right
 - both

Unspecified:

At this time, it is not specified how far up, left, down, etc. that the robot is pointing. However, it may be useful to consider pointing at objects approximately 1 or 2 feet away from the robot. Thus, when pointing down, for example, it should point at an object on the same surface as the robot but 1 to 2 feet in front of it.

Example:

```
{
    "name": "PointAt",
```

```
    "args": ["upwards"]
}
```

TiltHead

The TiltHead action instructs the robot to tilt its head to one side or the other.

List of arguments:

1. The direction that the head is to tilt. The possible directions are left and right, and are relative to the robot.
2. The amount of tilt in the robot's head. The head tilt may be either small or large.

Nao: This action is not supported by the Nao since the head does not move that way. Including this action in an Action Script for the Nao will result in the robot doing nothing and immediately moving on to the next action in the Script.

Example:

```
{
    "name": "TiltHead",
    "args": ["left", "small"]
}
```

SetEyes

The SetEyes action instructs the robot to change its eyes/face to express a particular emotion.

List of arguments:

1. The emotion that is to be expressed. It may be one of the following:
 - happy
 - confused
 - thinking
 - scared
 - terrified
 - love
 - looking
 - default

Nao: The Nao only supports changing the color of the eyes and which of the LEDs around the eye are turned on. Red eyes may be used for scared or terrified and blue eyes for confused. All other values can result in the robot using the standard light color for the eyes.

Note: This action will eventually be replaced with an ExpressEmotion action. The exhibited behavior for the robot may be specific to the modalities available in a robot. For example, the Misty can change its face, but a Nao can move its entire body.

Example:

```
{  
    "name": "SetEyes",  
    "args": ["happy"]  
}
```

Pause

The Pause action causes a momentary delay before the next action in the Action Script is performed. For most robots, each action is a non-blocking call, meaning that the next action can be executed even though the previous action has not yet completed. The Pause action provides time for the previous action to complete before moving on to the next action.

List of arguments:

1. Amount of time (in milliseconds) to pause

Example:

```
{  
    "name": "Pause",  
    "args": ["1500"]  
}
```